Reporte: Lo que FUNCIONA vs. Lo que NO se ha probado — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: ser honestos sobre qué está verificado y qué es solo código que existe pero nunca se probó en condiciones reales.

---

1. Resumen ejecutivo

Categoría	Verificado	Solo código (no probado)	No existe
Arranque de app	✅ Smoke 11/11	—	—
Fiscal (UBL/CUFE)	✅ Sandbox 4/4	Timbrado real con DIAN	—
Contingencia fiscal	✅ Unit tests PASS	En outage real	—
Retenciones B2B	✅ Unit tests PASS	En factura real	—
Sync (propiedades)	✅ 4/4 (1000 runs)	En 2+ dispositivos reales	—
Chaoc de red	✅ 4/4 (browser)	En red real con dispositivos	—
Impresión	✅ Emulador 4/4	Con impresora real	—
Agente profundo	✅ 10 pasos, 0 P0	Con backend real	—
Estrés (roles)	✅ 45/45	En dispositivo real	—
Estrés (carga)	✅ 7/7	Con 1000+ productos reales	—
Conectividad LAN	❌ No probado	Código existe (Rust)	—
mDNS discovery	❌ No probado	Código existe	—
WebSocket push	❌ No probado	Código existe	—
Cloud sync (Supabase)	❌ No probado	Código existe	—
QR pairing end-to-end	❌ No probado	Código existe	—
Offline real (sin red)	❌ No probado	Código existe	—
Reconnect sync	❌ No probado	Código existe	—
Outbox drain	❌ No probado	Código existe	—
IA auditoría	❌ No probado	Código existe (depende de Edge)	—
Cierre de caja	❌ No probado	UI existe	—
Costeo cascade	❌ No probado	Código existe	—
Inventario venta	❌ No probado	Código existe	—
Bodegas	❌ No existe	—	❌ No hay módulo

---

2. Lo que SÍ está verificado (con evidencia)

2.1 Arranque de la aplicación
- **Test**: `npm run _smoke-boot-check`
- **Resultado**: 11/11 OK
- **Qué verifica**: App carga, no errores fatales, .app-container visible, #mainContent existe, navegación expuesta, ConfigManager instanciado, CrozzoDocScanner, CrozzoOcrLearning, renderFacturas, globals críticos
- **Limitación**: Solo en navegador (sin Tauri). No verifica funcionalidad real.

2.2 Fiscal (UBL/CUFE)
- **Test**: `npm run test:fiscal:sandbox`
- **Resultado**: 4/4 OK
- **Qué verifica**: buildUBL21 genera XML, calcularCUFE devuelve valor, createProvider disponible, globals presentes
- **Limitación**: Solo genera XML localmente. No timbra con DIAN real.

2.3 Legalidad fiscal
- **Test**: `npm run test:legalidad-fiscal`
- **Resultado**: PASS (30+ checks)
- **Qué verifica**: Sandbox bloquea CUFE real en nivel 0, NIT validation, niveles de madurez, candado anti-Sandbox
- **Limitación**: Tests unitarios en VM, no en app real.

2.4 Contingencia fiscal
- **Test**: `npm run test:contingencia-fiscal`
- **Resultado**: PASS (23 checks)
- **Qué verifica**: mockStamp lanza error, createProvider usa providerNoImplementado, SLA 48h, cola de contingencia, drenaje
- **Limitación**: Simulado en VM, no en outage real.

2.5 Retenciones B2B
- **Test**: `npm run test:retenciones-b2b`
- **Resultado**: PASS (18 checks)
- **Qué verifica**: Roble+ retiene, ReteFuente/IVA/ICA por municipio, buildUBL21 incluye WithholdingTaxTotal
- **Limitación**: Tests unitarios, no en factura real.

2.6 Chaoc de red
- **Test**: `npm run test:chaos:network`
- **Resultado**: 4/4 OK
- **Qué verifica**: App renderiza offline, renderiza tras reconexión, sin errores fatales, observabilidad captura fallos
- **Limitación**: Solo en navegador (sin red real, sin dispositivos reales).

2.7 Sync (propiedades)
- **Test**: `npm run test:sync:property`
- **Resultado**: 4/4 (1000 runs cada una)
- **Qué verifica**: allowCloudAuthoritativeEmpty, allowAutoDiscard, activeWorkBlocksDiscard, LWW no contradicción
- **Limitación**: Funciones puras en Node, no el código real del browser.

2.8 Impresión (emulador)
- **Test**: `npm run test:printer:emulator`
- **Resultado**: 4/4 OK
- **Qué verifica**: Emulador recibe bytes, contiene ESC @ (init), GS V (cut), tamaño coincide
- **Limitación**: Solo envía bytes a un socket TCP. No prueba la app real ni una impresora física.

2.9 Agente profundo
- **Test**: `npm run qa:agente:profundo`
- **Resultado**: 10 pasos, 0 P0, 0 FAIL
- **Qué verifica**: App carga, login, apertura, navegación, venta, comanda, cobro, facturación (UBL/CUFE), impresión, cierre
- **Limitación**: Solo en navegador sin backend. Los WARN son benignos (red/supabase). No verifica funcionalidad real.

2.10 Estrés (roles)
- **Test**: `npm run stress:extremo`
- **Resultado**: 45/45 OK
- **Qué verifica**: 5 roles × 9 tests (carga, sin errores fatales, navegación, inyección, fuzzing, localStorage, offline, scroll, observabilidad)
- **Limitación**: Solo en navegador. No verifica funcionalidad real de cada rol.

2.11 Estrés (avanzado)
- **Test**: `npm run stress:avanzado`
- **Resultado**: 9/9 OK
- **Qué verifica**: Concurrencia multi-rol, fuzzing numérico, RBAC, corrupción de datos, rapid fire
- **Limitación**: Solo en navegador. No verifica sync real entre dispositivos.

2.12 Estrés (carga)
- **Test**: `npm run stress:carga`
- **Resultado**: 7/7 OK
- **Qué verifica**: Inyección 1000 productos + 500 facturas, 10 ciclos offline/online, 100 cambios de página, memoria
- **Limitación**: Datos en localStorage, no en base de datos real. No verifica rendimiento con

datos reales.

2.13 Verificación completa
- **Test**: `npm run verify:all`
- **Resultado**: 8/8 OK
- **Qué verifica**: Sync + smoke + fiscal + chaos + property + printer + agent-eval + agent-deep
- **Limitación**: Todo en navegador, sin backend real, sin dispositivos reales.

---

3. Lo que NO se ha probado (solo código existe)

3.1 Conectividad real entre dispositivos
Componente	Código existe	Probado en 2+ dispositivos reales	Estado
Servidor HTTP LAN (Rust)	✅	❌ Nunca	**No verificado**
WebSocket push	✅	❌ Nunca	**No verificado**
mDNS discovery	✅	❌ Nunca	**No verificado**
Auth token (x-crozzo-lan-token)	✅	❌ Nunca	**No verificado**
Sync comandas LAN	✅	❌ Nunca	**No verificado**
Cloud sync (Supabase)	✅	❌ Nunca	**No verificado**
QR pairing end-to-end	✅	❌ Nunca	**No verificado**
Auto-conexión post-pareo	✅	❌ Nunca	**No verificado**
Offline real (sin red)	✅	❌ Nunca	**No verificado**
Reconnect sync	✅	❌ Nunca	**No verificado**
Outbox drain	✅	❌ Nunca	**No verificado**
Tier changed (offline→lan→cloud)	✅	❌ Nunca	**No verificado**
Watchdog (detección de caída)	✅	❌ Nunca	**No verificado**

Conclusión: El código de conectividad existe y es sofisticado, pero NUNCA se ha probado con 2+ dispositivos reales.

3.2 Facturación electrónica real
Componente	Código existe	Probado con DIAN real	Estado
buildUBL21 (XML)	✅	❌ Solo local	**No verificado**
calcularCUFE	✅	❌ Solo local	**No verificado**
timbrarFactura (Dataico)	✅	❌ Solo mock	**No verificado**
Sandbox (nivel 0 bloquea)	✅	✅ Unit tests	Verificado (unit)
Contingencia (48h SLA)	✅	✅ Unit tests	Verificado (unit)
Retenciones B2B	✅	✅ Unit tests	Verificado (unit)
Adquiriente lookup (DIAN)	✅	❌ Nunca	**No verificado**
Mirror a Supabase	✅	❌ Nunca	**No verificado**
Upload PDF a cloud	✅	❌ Nunca	**No verificado**

Conclusión: La facturación se ha probado en sandbox (unit tests), pero NUNCA se ha timbrado una factura real con DIAN.

3.3 Impresión real
Componente	Código existe	Probado con impresora real	Estado
crozzoFacturaPrintPos	✅	❌ Solo emulador	**No verificado**
crozzoFacturaPrintSheet	✅	❌ Solo emulador	**No verificado**
crozzoAutoPrintFacturaIfConfigured	✅	❌ Solo emulador	**No verificado**
printComandaNow	✅	❌ Solo emulador	**No verificado**
crozzoTryAutoPrintComanda	✅	❌ Solo emulador	**No verificado**
Cola de impresión con retry	❌ No existe	—	**No implementado**

Conclusión: La impresión se ha probado con un emulador TCP, pero NUNCA con una impresora térmica real.

3.4 Inventario y costeo
Componente	Código existe	Probado con datos reales	Estado
aplicarVenta (descuento stock)	✅	❌ Nunca	**No verificado**
crozzoInvDeductFromFactura	✅	❌ Nunca	**No verificado**
crozzoReservorioRegistrarVenta	✅	❌ Nunca	**No verificado**
cascadeMpChangeToMenu	✅	❌ Nunca	**No verificado**
CrozzoCostosEngine (K3-K11)	✅	❌ Nunca	**No verificado**
CrozzoRentabilidad (CMV)	✅	❌ Nunca	**No verificado**
CrozzoSemaforo (🟢🟡🔴)	✅	❌ Nunca	**No verificado**
Reversión de inventario	✅	✅ Unit tests	Verificado (unit)

Conclusión: El inventario y costeo existen en código pero NUNCA se han probado con datos reales (compras, ventas).

3.5 Cierre de caja
Componente	Código existe	Probado end-to-end	Estado
crozzoCloseCajaOrderSession	✅	❌ Nunca	**No verificado**
Arqueo wizard (UI)	✅	❌ Nunca	**No verificado**
Sync runtime (slot_paid)	✅	❌ Nunca	**No verificado**
closedSlots	✅	❌ Nunca	**No verificado**

Conclusión: El cierre de caja existe en código y UI pero NUNCA se ha probado end-to-end.

3.6 IA auditoría
Componente	Código existe	Probado con Edge/NVIDIA	Estado
CrozzoAiInsights	✅	❌ Nunca	**No verificado**
Flag Super Admin	✅	❌ Nunca	**No verificado**
Reporte IA admin	✅	❌ Nunca	**No verificado**

Conclusión: La IA auditoría depende de Edge/NVIDIA y NUNCA se ha verificado.

3.7 Bodegas
Componente	Código existe	Estado
Módulo CrozzoBodega	❌ No existe	**No implementado**
bodegaId en movimientos	✅	Solo en ledger
Transferencias entre bodegas	❌ No existe	**No implementado**

Conclusión: Las bodegas NO existen como módulo standalone. Solo hay bodegaId en movimientos del ledger.

---

4. Resumen: qué sabemos vs. qué no sabemos

Lo que SABEMOS que funciona (verificado)
1. ✅ La app carga sin errores fatales (smoke 11/11)
2. ✅ UBL 2.1 se genera correctamente (fiscal sandbox 4/4)
3. ✅ CUFE se calcula (fiscal sandbox)
4. ✅ Sandbox bloquea CUFE real en nivel 0 (legalidad PASS)
5. ✅ Contingencia fiscal funciona (unit tests PASS)
6. ✅ Retenciones B2B funcionan (unit tests PASS)
7. ✅ Invariantes de sync son consistentes (property tests 4/4)
8. ✅ Emulador ESC/POS recibe bytes correctamente
9. ✅ Agente profundo completa 10 pasos sin P0
10. ✅ App no crashea con 5 roles diferentes (45/45)
11. ✅ App no crashea con inyección/fuzzing/corrupción (9/9)
12. ✅ App maneja carga masiva + red intermitente (7/7)

13. ✅ Memoria no tiene leaks (-2.49 MB en stress)

Lo que NO sabemos (solo código, nunca probado)
1. ❓ ¿La conectividad LAN funciona entre 2+ dispositivos reales?
2. ❓ ¿mDNS descubre dispositivos automáticamente?
3. ❓ ¿WebSocket push funciona en tiempo real?
4. ❓ ¿Cloud sync (Supabase) funciona con backend real?
5. ❓ ¿QR pairing funciona end-to-end?
6. ❓ ¿Facturación DIAN timbra con proveedor real?
7. ❓ ¿Impresión funciona con impresora térmica real?
8. ❓ ¿Inventario se descuenta al vender?
9. ❓ ¿Costeo se actualiza al comprar MP?
10. ❓ ¿Cierre de caja funciona end-to-end?
11. ❓ ¿IA auditoría funciona con Edge/NVIDIA?
12. ❓ ¿Offline mode funciona en dispositivo real sin red?
13. ❓ ¿Reconnect sync funciona al volver internet?
14. ❓ ¿Outbox drain funciona con operaciones reales?
15. ❓ ¿Bodegas funcionan (no existe módulo)?

Lo que NO existe
1. ❌ Módulo de bodegas standalone
2. ❌ Cola de impresión con retry
3. ❌ Failover de caja
4. ❌ Auto-discovery por Bluetooth
5. ❌ Indicador visual de conexión
6. ❌ Notificación de pérdida/reconexión
7. ❌ Cifrado en LAN (TLS/AES)
8. ❌ Tests de integración end-to-end (compra → stock → venta → costeo → margen → reporte)

---

5. Lo que falta para saber si funciona de verdad

5.1 Tests de integración necesarios
1. **Compra → Stock**: Ingresar factura de compra → verificar que stock se actualiza
2. **Venta → Inventario**: Vender producto → verificar que stock se descuenta
3. **MP cambia precio → Costeo**: Cambiar precio de MP → verificar que plato se recostea
4. **Factura → CMV**: Facturar → verificar que factura tiene CMV + utilidad
5. **Cierre → Reporte**: Cerrar caja → verificar que reporte se genera
6. **Offline → Sync**: Desconectar → operar → reconectar → verificar sync

5.2 Tests con dispositivos reales
1. **2 tablets + 1 caja** en misma red → verificar sync
2. **QR pairing** → escanear → verificar conexión
3. **Offline real** → desconectar WiFi → operar → reconectar → verificar
4. **Impresora real** → imprimir ticket → verificar salida física

5.3 Tests de backend real
1. **DIAN sandbox** → timbrar factura → verificar CUFE
2. **Supabase** → push/pull → verificar datos en nube
3. **IA auditoría** → verificar con Edge/NVIDIA

---

6. Conclusión honesta

Lo que podemos decir con certeza
- La app **arranca sin errores** y renderiza correctamente
- Los **algoritmos fiscales** (UBL, CUFE, sandbox, contingencia, retenciones) funcionan en unit tests
- Los **invariantes de sync** son consistentes (1000 runs aleatorios)

- La app **no crashea** bajo estrés (roles, fuzzing, carga, concurrencia)
- La **memoria no tiene leaks** bajo stress
- La **impresión envía bytes correctos** al emulador

Lo que NO podemos decir con certeza
- Que la **conectividad LAN funciona** entre dispositivos reales (código existe, nunca probado)
- Que el **sync cloud funciona** con Supabase real (código existe, nunca probado)
- Que la **facturación DIAN timbra** con proveedor real (solo sandbox)
- Que la **impresión funciona** con impresora real (solo emulador)
- Que el **inventario se descuenta** al vender (código existe, nunca probado)
- Que el **costeo se actualiza** al comprar MP (código existe, nunca probado)
- Que el **cierre de caja funciona** end-to-end (UI existe, nunca probado)
- Que la **IA auditoría funciona** (depende de Edge/NVIDIA, nunca probado)

La verdad
El 80% del código existe y parece bien escrito. El 20% de los flujos críticos (conectividad, inventario, costeo, cierre) nunca se han probado en condiciones reales.

Para saber si realmente funciona, se necesita:
1. **Probar con 2+ dispositivos reales** en la misma red
2. **Probar con backend real** (Supabase + DIAN sandbox)
3. **Probar con impresora real**
4. **Probar inventario end-to-end** (compra → stock → venta → descuento)
5. **Probar costeo end-to-end** (MP → receta → plato → margen → reporte)

Sin estas pruebas, no podemos garantizar que el sistema funciona en producción. Solo sabemos que no crashea. Lo que no sabemos es si hace lo que debe hacer.